

AX 환경을 위한 ERP 솔루션 기반

멀티 하이브리드 클라우드 인프라 자동화 및 Observability 체계 구축

김진우

4인 팀 프로젝트 (팀명: Siseon)

인프라 설계 · 구축 · CI/CD 담당

2026.03 ~ 2026.06

<https://github.com/jinuuuKim/Stockops-Infra>

<https://github.com/jinuuuKim/Stockops-GitOps>

INDEX



01 개요

1. 시나리오 및 설계 기준
2. 전체 아키텍처
3. 담당 영역



02 구현

1. 인프라 자동화 및 멀티 리전 설계
2. 네트워크 구조 / 네트워크 리소스
3. 트래픽 분리 / Health Check
4. EKS 구성 / HPA + Karpenter
5. CI/CD 파이프라인
6. IoT 파이프라인 / 센서 데이터 실시간 처리·이력 저장
7. 키리스 인증
8. 시크릿 관리
9. 경계 방어 - 애플리케이션 계층(WAF) + 네트워크 계층(SG)



03 결과

1. 트러블슈팅
2. 성과

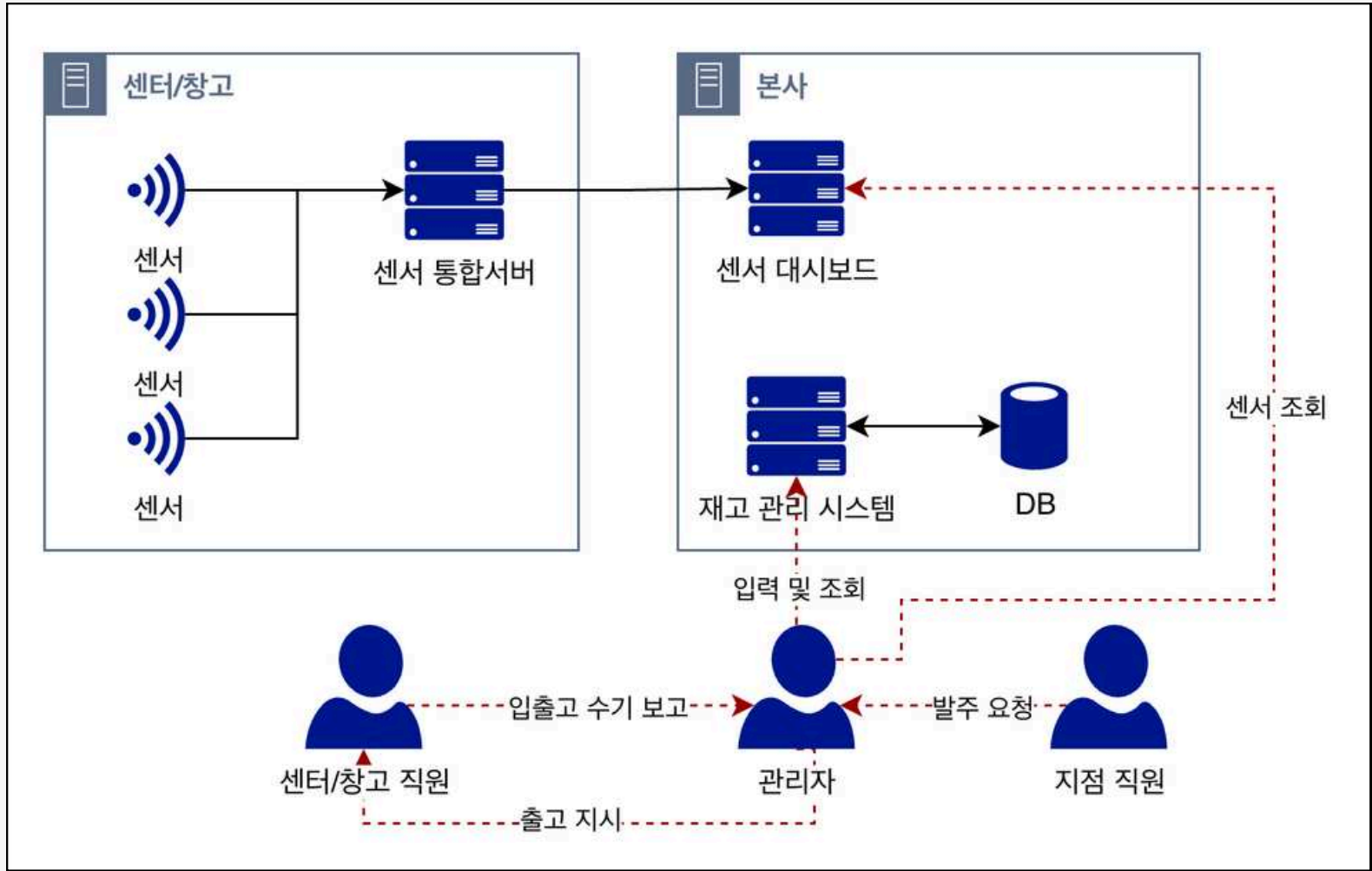
시나리오 및 설계 기준

물류센터 센서 데이터를 실시간 수집하고, 한국 본사·미국 영업팀이 웹에서 주문·재고를 관리하는 ERP/WMS 서비스

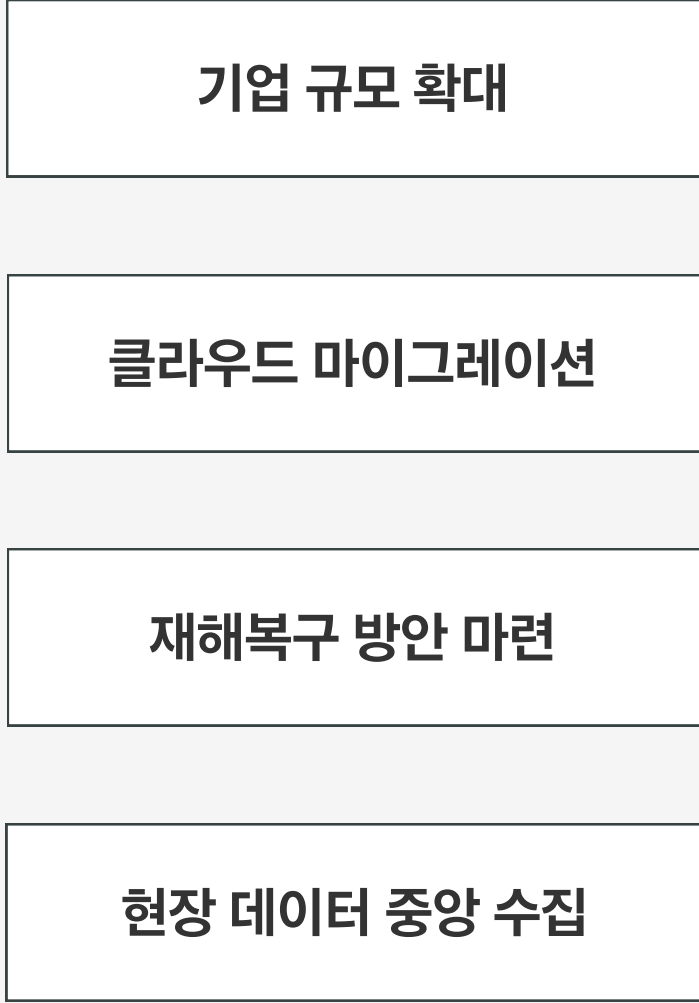
기획 배경

- 온프레미스 ERP를 운영하던 K-Food 수출 기업의 미국 진출을 가정한 시나리오
- 국내 기업의 클라우드 전환 사례와 단일 리전 장애 사례를 참고해 글로벌 확장과 재해복구를 설계 축으로 설정
- 전체 구성을 코드로 재현 가능하게 만드는 것을 목표로 함

기존 기업 구조



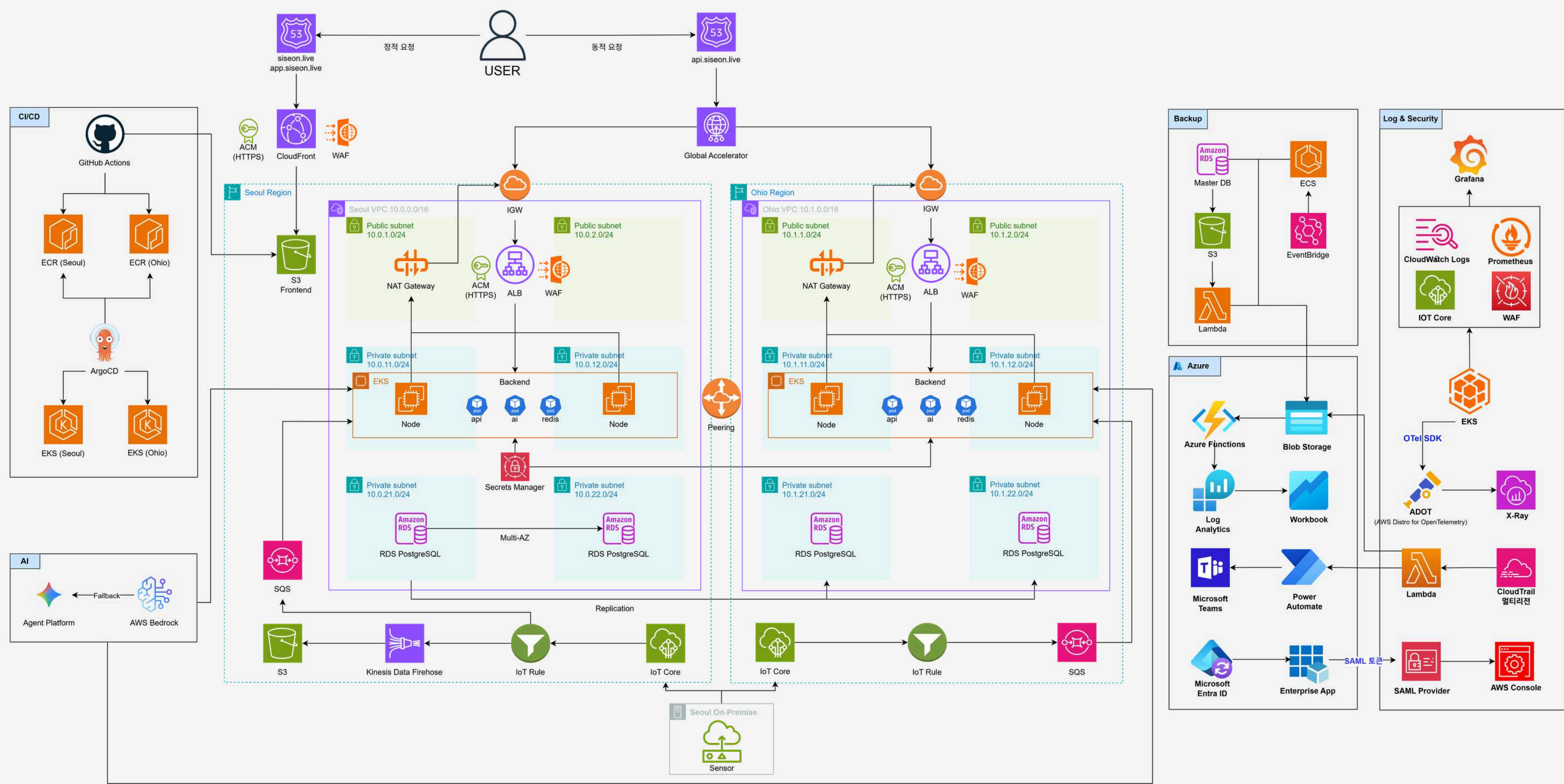
요구사항

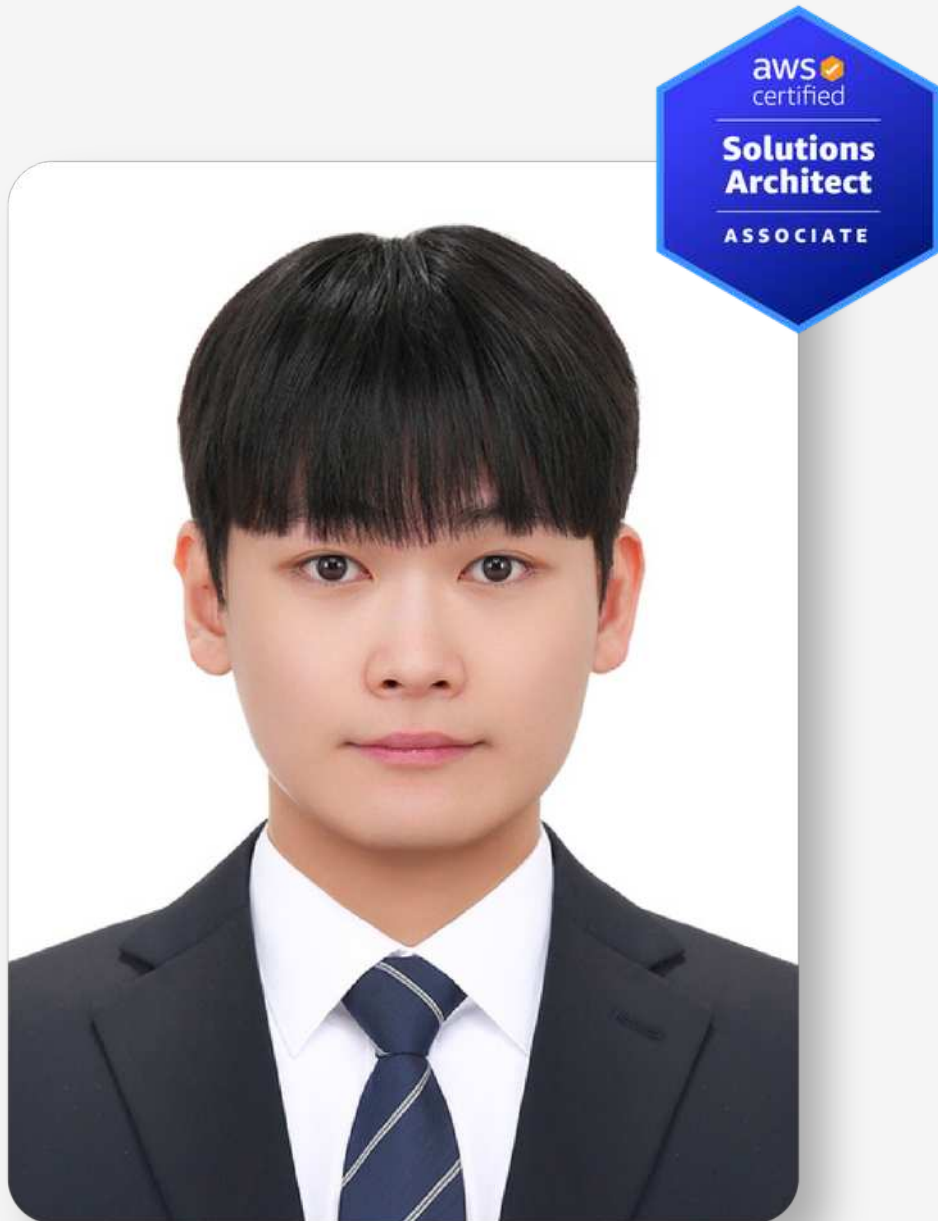


설계 대응



전체 아키텍처





김진우

AWS 인프라 설계 및 구축
Terraform 기반 IaC 자동화
CI/CD

담당 영역

- 1 멀티 리전 인프라 자동화 및 네트워크 설계
- 2 정적 / 동적 트래픽 분리
- 3 EKS + Auto Scaling
- 4 CI / CD 파이프라인
- 5 IoT 센서 파이프라인
- 6 인프라 보안 설계

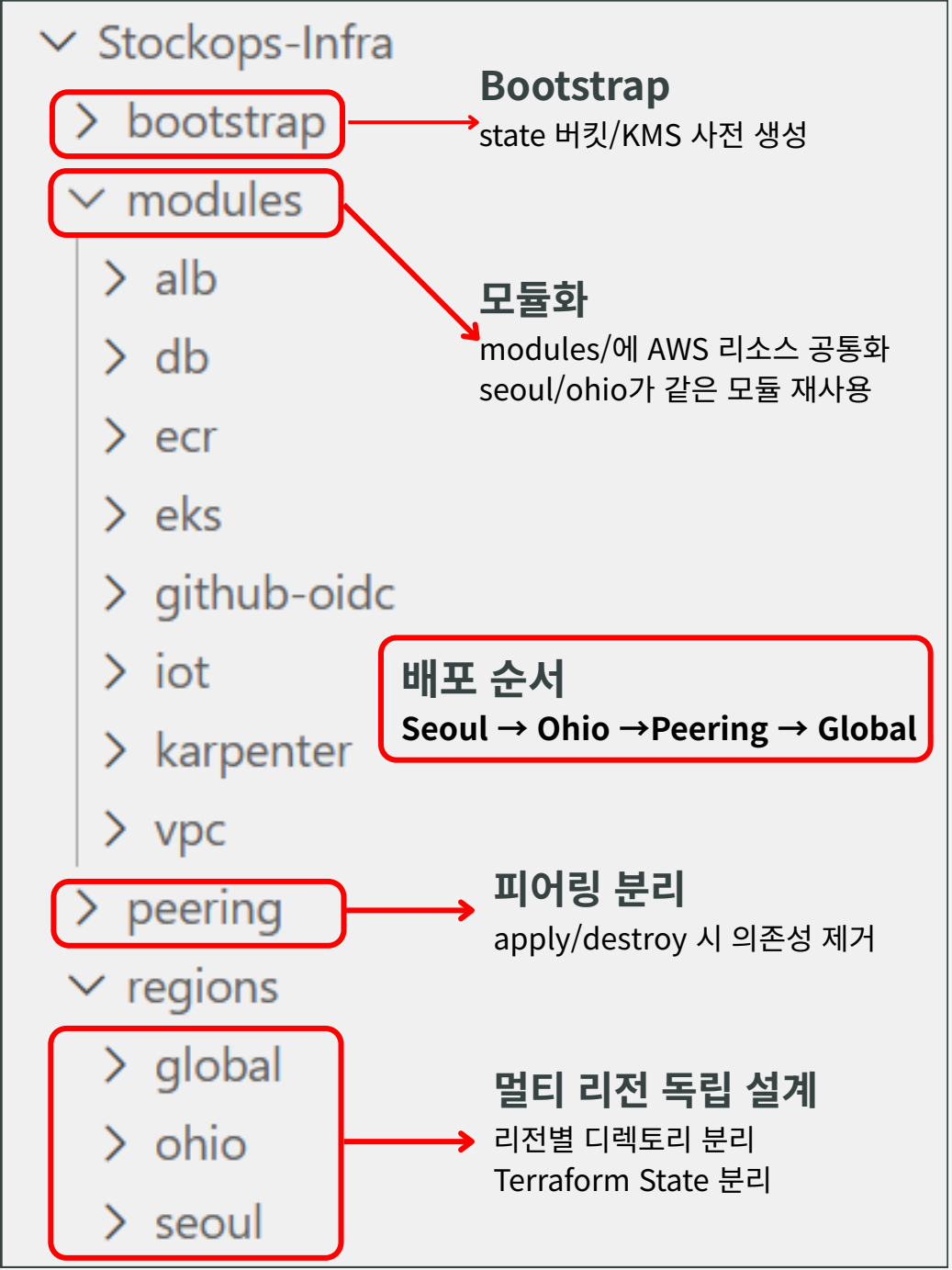
사용 서비스 및 기술

- IaC Terraform · Git
- AWS VPC · ALB · Global Accelerator · CloudFront · Route53 · EKS · EC2 · RDS · S3 · ECR · ACM · WAF · IAM · Secrets Manager · IoT Core · SQS · Kinesis Firehose · CloudWatch
- Container Kubernetes · Docker · Karpenter · Helm · ESO · LBC
- CI/CD GitHub Actions · ArgoCD

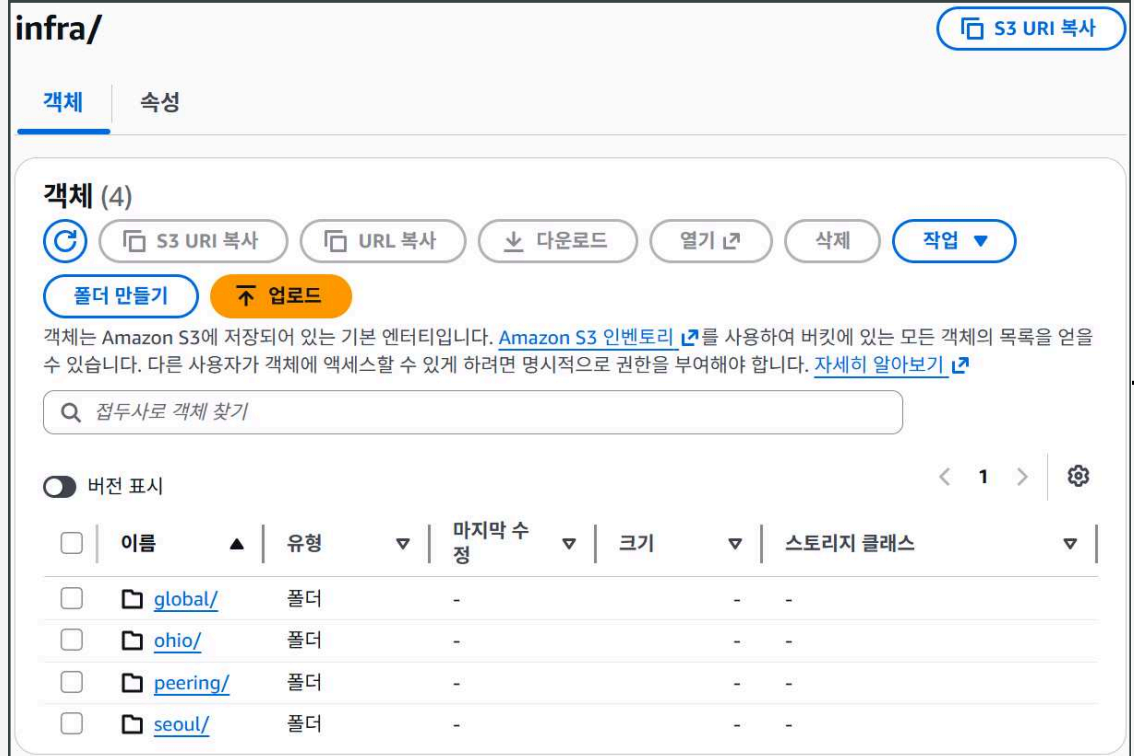
인프라 자동화 및 멀티 리전 설계

인프라 전 구간을 Terraform으로 코드화, State를 리전·계층 기준으로 4분리

Terraform 디렉토리 구조



Terraform State → S3



provider.tf

```
backend "s3" {  
  bucket      = "siseon-terraform-state"  
  key         = "infra/seoul/terraform.tfstate"  
  region      = "ap-northeast-2"  
  profile     = "siseon"  
  encrypt     = true  
  kms_key_id  = "alias/siseon-tfstate"  
  use_lockfile = true  
}
```

1 Terraform State 파일 분리

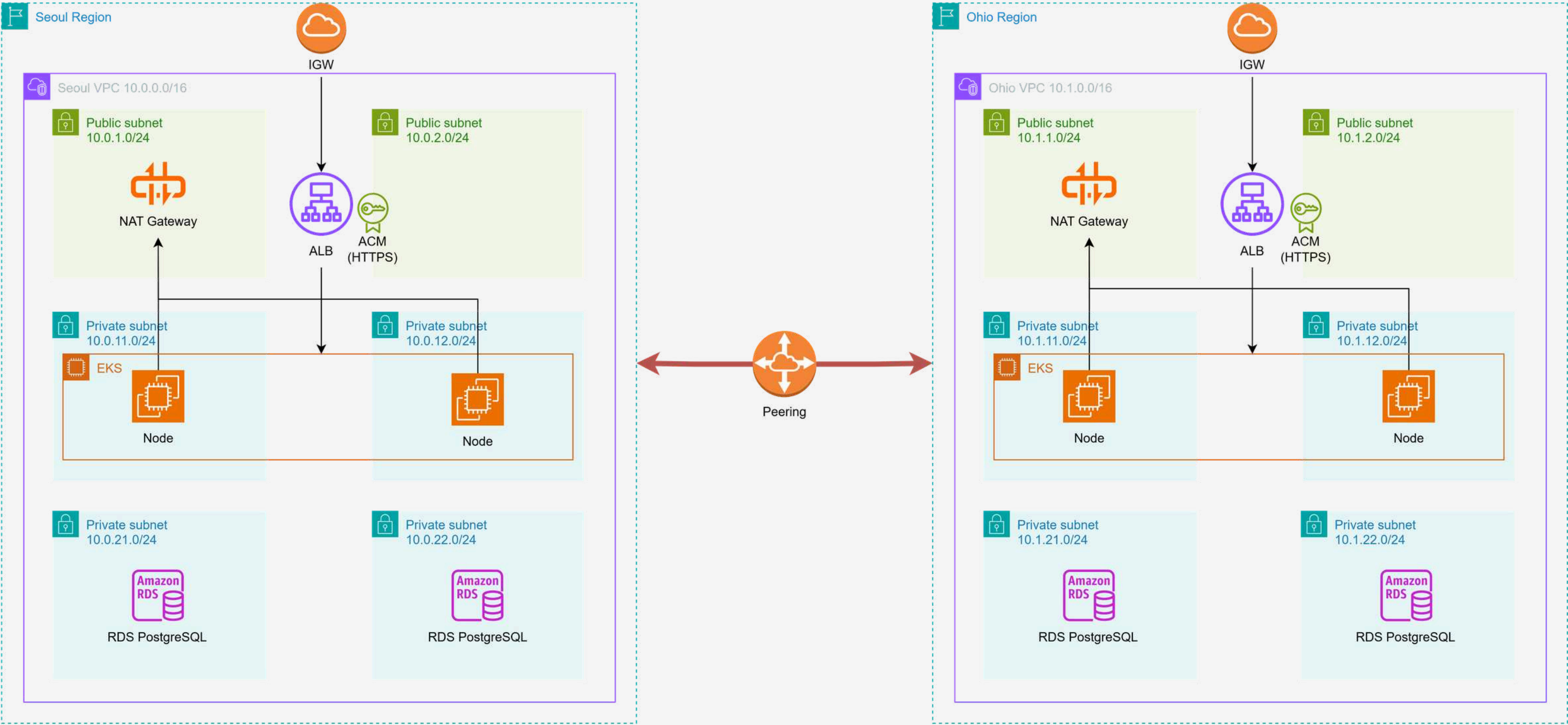
- 리전별(seoul/ohio/global/peering) Terraform State 파일 분리
- Terraform State 파일은 S3 버킷(siseon-terraform-state/infra)에 저장
- Seoul/Ohio가 독립적으로 apply/destroy 되기 위해 Peering State 별도 관리

2 암호화 및 Lock

- S3에 SSE 활성화
- 고객 관리 KMS 키(CMK)로 암호화
- S3 네이티브 락 → DynamoDB 없이 동시 apply 차단

네트워크 구조

3 Tier로 나눠 외부에서 DB에 직접 접근 차단



네트워크 리소스

VPC·서브넷·라우팅 테이블·NAT·IGW를 두 리전에 동일 구조로 생성

VPC

ohio-vpc	vpc-05169f98e637f288b	✔ Available
seoul-vpc	vpc-01d909e59f32bb895	✔ Available

Seoul-Ohio Peering

Name	피어링 연결 ID	상태
seoul-ohio-peering	pcx-01ebe6284ae00b678	✔ 활성

Subnet

Name	서브넷 ID	상태	VPC
ohio-pub-sub-2c	subnet-05725748fd57f31e7	✔ Available	vpc-05169f98e637f288b
ohio-pub-sub-2a	subnet-0eb68e93adc3a7593	✔ Available	vpc-05169f98e637f288b
ohio-priv-db-2c	subnet-0cb508683fe71f6af	✔ Available	vpc-05169f98e637f288b
ohio-priv-db-2a	subnet-073093ad851c9565b	✔ Available	vpc-05169f98e637f288b
ohio-priv-app-2c	subnet-046c6dd80c98f1ca4	✔ Available	vpc-05169f98e637f288b
ohio-priv-app-2a	subnet-00a9f6592b5f710b3	✔ Available	vpc-05169f98e637f288b

Name	서브넷 ID	상태	VPC
seoul-pub-sub-2c	subnet-0eaf057389746747d	✔ Available	vpc-01d909e59f32bb895
seoul-pub-sub-2a	subnet-0a2a1fa12d097fa26	✔ Available	vpc-01d909e59f32bb895
seoul-priv-db-2c	subnet-08464949bd8d49354	✔ Available	vpc-01d909e59f32bb895
seoul-priv-db-2a	subnet-06fe59b530d01013a	✔ Available	vpc-01d909e59f32bb895
seoul-priv-app-2c	subnet-0e21f99bdaac615ab	✔ Available	vpc-01d909e59f32bb895
seoul-priv-app-2a	subnet-01ca6f76dde3b7a41	✔ Available	vpc-01d909e59f32bb895

Routing Table

Name	라우팅 테이블 ID	명시적 서브넷 연결
seoul-pub-rt	rtb-0cc7564c57c396b3c	2 서브넷
seoul-priv-db-rt	rtb-0a1af4f5fb6e8e06e	2 서브넷
seoul-priv-app-rt	rtb-041f8064ef665412f	2 서브넷

Name	라우팅 테이블 ID	명시적 서브넷 연결
ohio-pub-rt	rtb-02632a739f7092f67	2 서브넷
ohio-priv-db-rt	rtb-0c591d6d25402a829	2 서브넷
ohio-priv-app-rt	rtb-0e74d86642a7e5e9f	2 서브넷

NAT Gateway

Name	NAT 게이트웨이 ID	연결 유형	상태
seoul-nat-gw	nat-01255c7a614e3ed09	Public	✔ Available

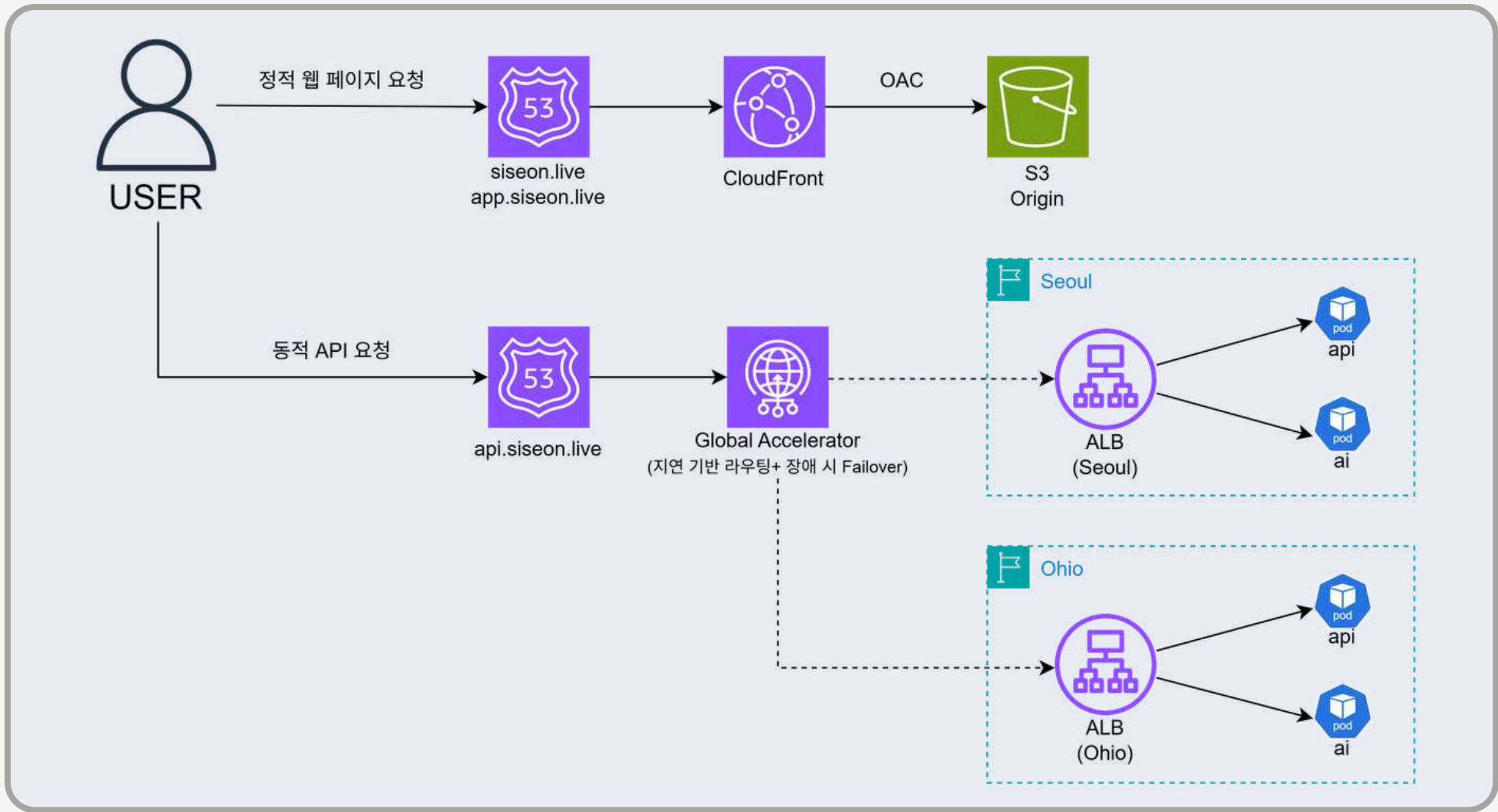
Name	NAT 게이트웨이 ID	연결 유형	상태
ohio-nat-gw	nat-0c9868849dd6c3da2	Public	✔ Available

IGW

Name	인터넷 게이트웨이 ID	상태
seoul-igw	igw-0d59706ac22609d32	✔ Attached
Name	인터넷 게이트웨이 ID	상태
ohio-igw	igw-0ece890cf7b1c3c41	✔ Attached

트래픽 분리

정적 트래픽은 CloudFront + S3, 동적 트래픽은 Global Accelerator → ALB → EKS로 경로 분리



Global Accelerator

액셀러레이터 (1)						
Q 액셀러레이터 찾기						
이름	유형	IPv4	IPv6	활성화됨	DNS 이름	
stockops-global-accelerator	스탠다드	166.117.90.183 166.117.152.156		활성화됨	a79403588ec9cbdb4.awsglobalaccelerator.com	

1 정적 트래픽

- S3에 빌드된 정적 파일(HTML/JS/CSS)을 업로드
- CloudFront가 엣지 캐싱으로 전 세계에 배포
- EKS를 거치지 않고 CloudFront 엣지에서 직접 응답

2 동적 트래픽

- GA 지연 기반 라우팅으로 가까운 리전으로 트래픽 전송
- AWS 백본 네트워크로 퍼블릭 인터넷 구간을 최소화
- 한리전 장애 시 자동 Failover로 가용성 확보

CloudFront Distribution

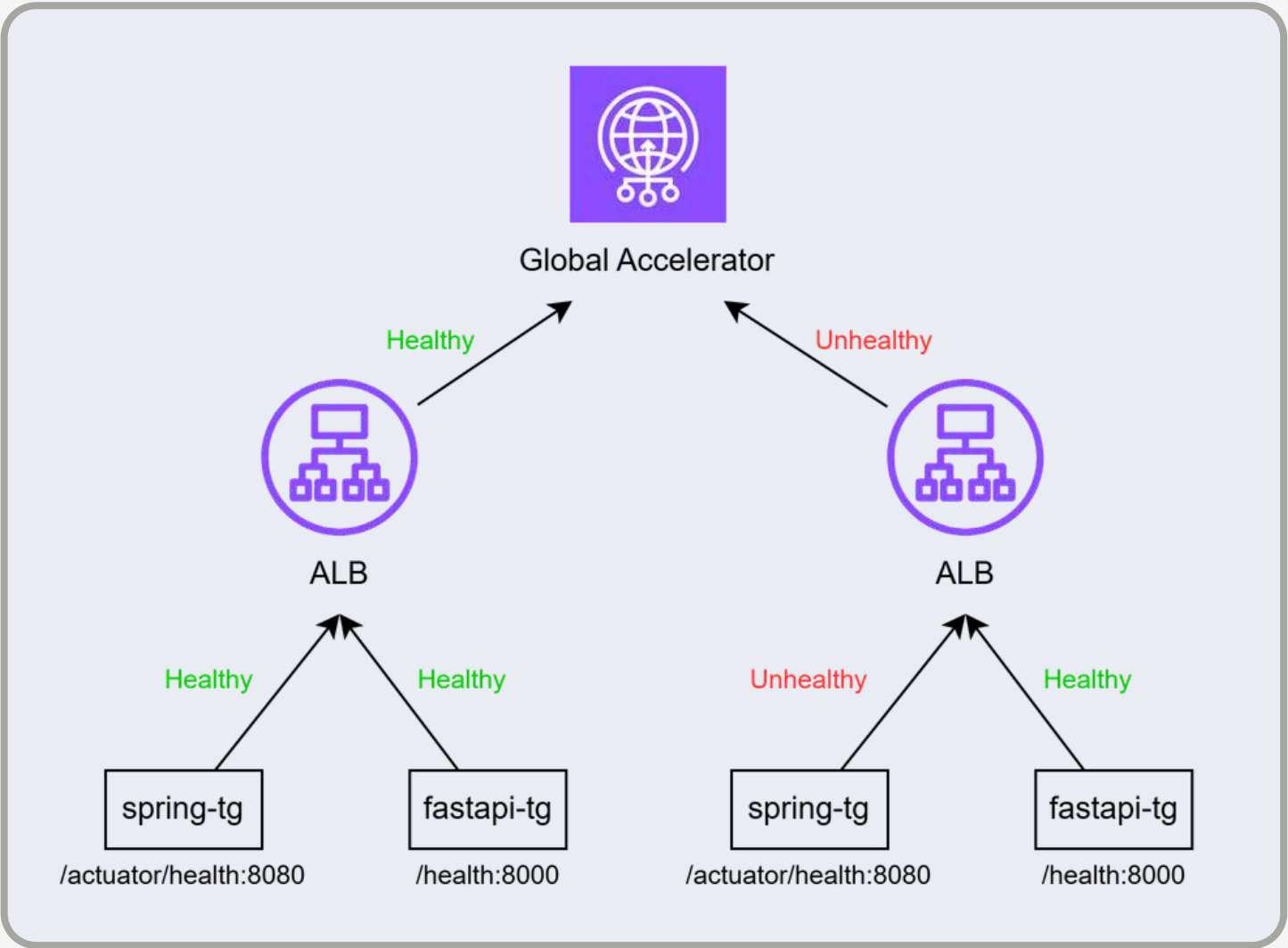
ID	상태	설명	유형	Domain name	대체 도메인 이름
EXOBRANYLVQ24	활성화됨	siseon client-web static	Standard	d3l3t08l...	siseon.live
E2UBZ3JOBFBKXC	활성화됨	siseon admin-web static	Standard	d45ur2m...	app.siseon.live

CloudFront OAC

원본 이름	원본 도메인	원본 경로	원본 유형	Origin ...	Origin access
s3-admin	siseon-frontend-admin.s3.ap-northeast-2.amazonaws.com		S3	-	E2MJZ3JGNKRO3Z
원본 이름	원본 도메인	원본 경로	원본 유형	Origin ...	Origin access
s3-client	siseon-frontend-client.s3.ap-northeast-2.amazonaws.com		S3	-	E2MJZ3JGNKRO3Z

Health Check

지연 기반 라우팅(한국→서울, 미국→오하이오) + 리전 장애 시 자동 페일오버



spring_tg (api-server)

IP 주소	▼	포트	▼	영역	▼	상태 확인
10.0.11.100		8080		ap-northeast-2a (apne2-az1)		🟢 Healthy
IP 주소	▼	포트	▼	영역	▼	상태 확인
10.1.11.174		8080		us-east-2a (use2-az1)		🟢 Healthy

fastapi_tg (ai-module)

IP 주소	▼	포트	▼	영역	▼	상태 확인
10.0.12.130		8000		ap-northeast-2c (apne2-az3)		🟢 Healthy
IP 주소	▼	포트	▼	영역	▼	상태 확인
10.1.11.90		8000		us-east-2a (use2-az1)		🟢 Healthy

Global Accelerator Endpoint

리스너 ID	▼	포트 및 프로토콜	▼	엔드포인트 그룹	▼	상태
arn...156b2219		리스너: 443 TCP		ap-northeast-2, us-east-2		🟢 모두 정상
arn...94f9a47e		리스너: 80 TCP		ap-northeast-2, us-east-2		🟢 모두 정상

Target Group, GA Endpoint 모두 Healthy

EKS 구성

두 리전 동일 클러스터 구성 - 관리형 노드 그룹 + Karpenter 혼합

seoul/ohio-cluster

클러스터 이름 ▲	상태 ▼	Kubernetes 버전 ▼	지원 기간
ohio-cluster	🟢 활성	1.30 지금 업그레이드	⚠️ 2026년 7월 23일까지 추가 지원
클러스터 이름 ▲	상태 ▼	Kubernetes 버전 ▼	지원 기간
seoul-cluster	🟢 활성	1.30 지금 업그레이드	⚠️ 2026년 7월 23일까지 추가 지원

Managed Node Group

Namespace	구성 요소
kube-system	Metrics Server, LBC
argocd	ArgoCD
karpenter	Karpenter
external-secrets	ESO
stockops	api-server, ai-module, redis

❶ Node 스펙

- t3.medium
- desired = 2
- min = 2
- max = 4

❷ App Pod

- api-server = 1
- ai-module = 1
- redis = 1 (고정)

Managed Node Group

```
PS C:\KJW\Team_Project\Stockops-Infra\regions\seoul> kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
ip-10-0-11-184.ap-northeast-2.compute.internal Ready    <none>   34m   v1.30.14-eks-93b80c6
ip-10-0-12-171.ap-northeast-2.compute.internal Ready    <none>   33m   v1.30.14-eks-93b80c6

PS C:\KJW\Team_Project\Stockops-Infra\regions\ohio> kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
ip-10-1-11-82.us-east-2.compute.internal Ready    <none>   21m   v1.30.14-eks-93b80c6
ip-10-1-12-89.us-east-2.compute.internal Ready    <none>   21m   v1.30.14-eks-93b80c6
```

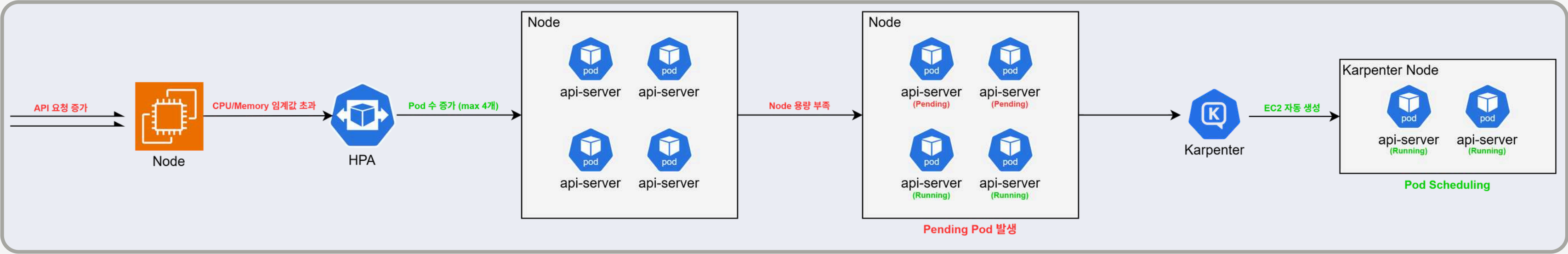
Backend Pod

```
PS C:\KJW\Team_Project\Stockops-Infra\regions\seoul> kubectl get pods -n stockops
NAME                                READY    STATUS    RESTARTS  AGE
stockops-ai-9b99f987-zlgh1         1/1      Running   0          24m
stockops-api-5c9cd57cf8-lnchj       1/1      Running   0          24m
stockops-redis-88d6cbdcf-qwzqr      1/1      Running   0          73m

PS C:\KJW\Team_Project\Stockops-Infra\regions\ohio> kubectl get pods -n stockops
NAME                                READY    STATUS    RESTARTS  AGE
stockops-ai-58dd99ccd9-shdfs        1/1      Running   0          29m
stockops-api-5cb4c9f494-2wbx2       1/1      Running   0          22m
stockops-redis-88d6cbdcf-dqjix      1/1      Running   0          59m
```

HPA + Karpenter

파드는 HPA, 노드는 Karpenter로 2단 Auto Scaling, 인스턴스는 Spot 우선



HPA 설정			
대상	min	max	기준
api	1	4	CPU 60%
ai	1	4	CPU 60% + Memory 70%

Karpenter 설정	
- 인스턴스: t3.medium / t3.large - 구매 옵션: Spot 우선, On-demand 혼합 - 유허 노드 자동 반납: consolidateAfter = 1 분	

HPA 확인

PS C:\KJW\Team_Project\Stockops-Infra\regions\seoul> kubectl get hpa -n stockops							
NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE	
stockops-ai-hpa	Deployment/stockops-ai	cpu: 0%/60%, memory: 21%/70%	1	4	1	84m	
stockops-api-hpa	Deployment/stockops-api	cpu: 11%/60%	1	4	4	84m	

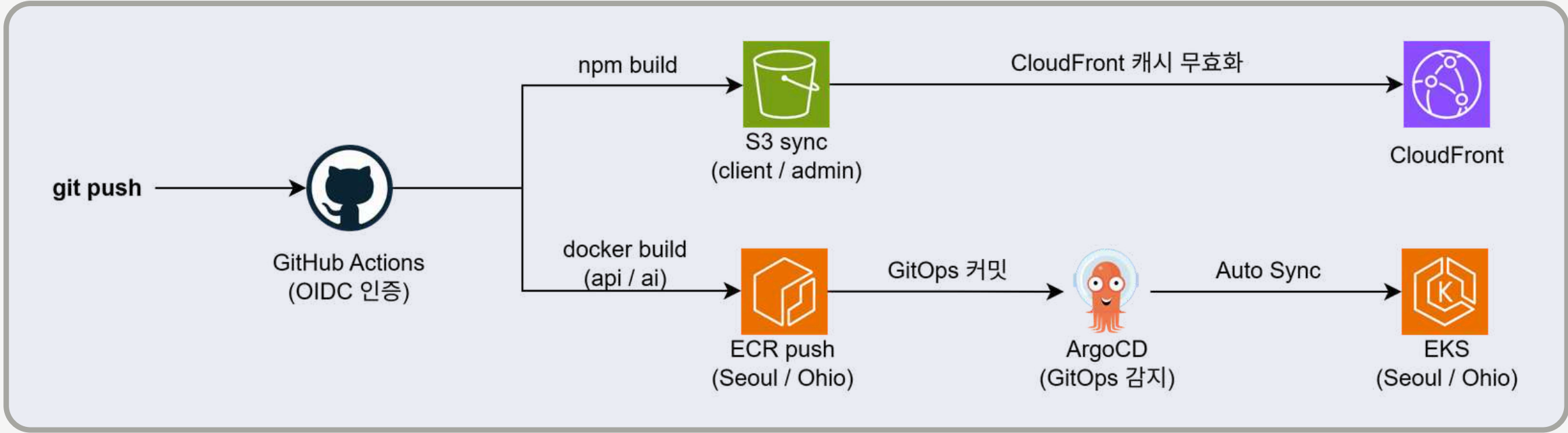
Karpenter node

PS C:\KJW\Team_Project\Stockops-Infra\regions\ohio> kubectl get nodeclaims							
NAME	TYPE	CAPACITY	ZONE	NODE	READY	AGE	
ohio-cluster-node-pool-6g9hq	t3.large	spot	us-east-2c		Unknown	23s	
ohio-cluster-node-pool-d2nws	t3.large	spot	us-east-2c		Unknown	30s	

PS C:\KJW\Team_Project\Stockops-Infra\regions\ohio> kubectl get nodepool					
NAME	NODECLASS	NODES	READY	AGE	
ohio-cluster-node-pool	ohio-cluster-node-class	2	True	59m	

CI/CD 파이프라인

코드 push부터 클러스터 반영까지 무인 자동화 (GitHub Actions + ArgoCD)



1 Frontend

클러스터를 거치지 않는 경로

- GitHub Actions가 client/admin 두 앱을 npm build로 빌드해 S3에 sync함
- 정적 파일이라 컨테이너 이미지가 불필요하고 EKS도 관여하지 않음
- CloudFront 캐시 무효화를 통해 옛지에 남은 이전 파일이 계속 서빙되는 것을 막음
- 파드를 건드리지 않아 Backend와 무관하게 반복 가능

GitHub Actions

Build & Deploy

succeeded on Jun 19 in 3m 58s

> Set up job

> Checkout

> Configure AWS Credentials

> Setup Node.js

> Build & Deploy client-web → S3

> Build & Deploy admin-web → S3

> Login to ECR

> Build & Push api-server

> Build & Push ai-module

> Setup Kustomize

> Update GitOps manifest (Seoul)

> Update GitOps manifest (Ohio)

> Invalidate CloudFront cache

ArgoCD

Applications / Q stockops-seoul

APPLICATION DETAILS TREE

DETAILS

DIFF

SYNC

SYNC STATUS

HISTORY AND ROLLBACK

DELETE

REFRESH

APP HEALTH

Healthy

SYNC STATUS

Synced to main (d30f358)

Auto sync is enabled.

Author: github-actions[bot] <github-actions[bot]@users.norep...

LAST SYNC

Sync OK to d30f358

Succeeded 11 minutes ago (Wed Jun 24 2026 14:40:25 GMT+0900)

Author: github-actions[bot] <github-actions[bot]@users.norep...

stockops-seoul

11 minutes

rev:1

stockops-ai

11 minutes

rev:1

stockops-api

11 minutes

rev:1

stockops-ai-687c7c6867

11 minutes

rev:1

stockops-api-798757cf4f

11 minutes

rev:1

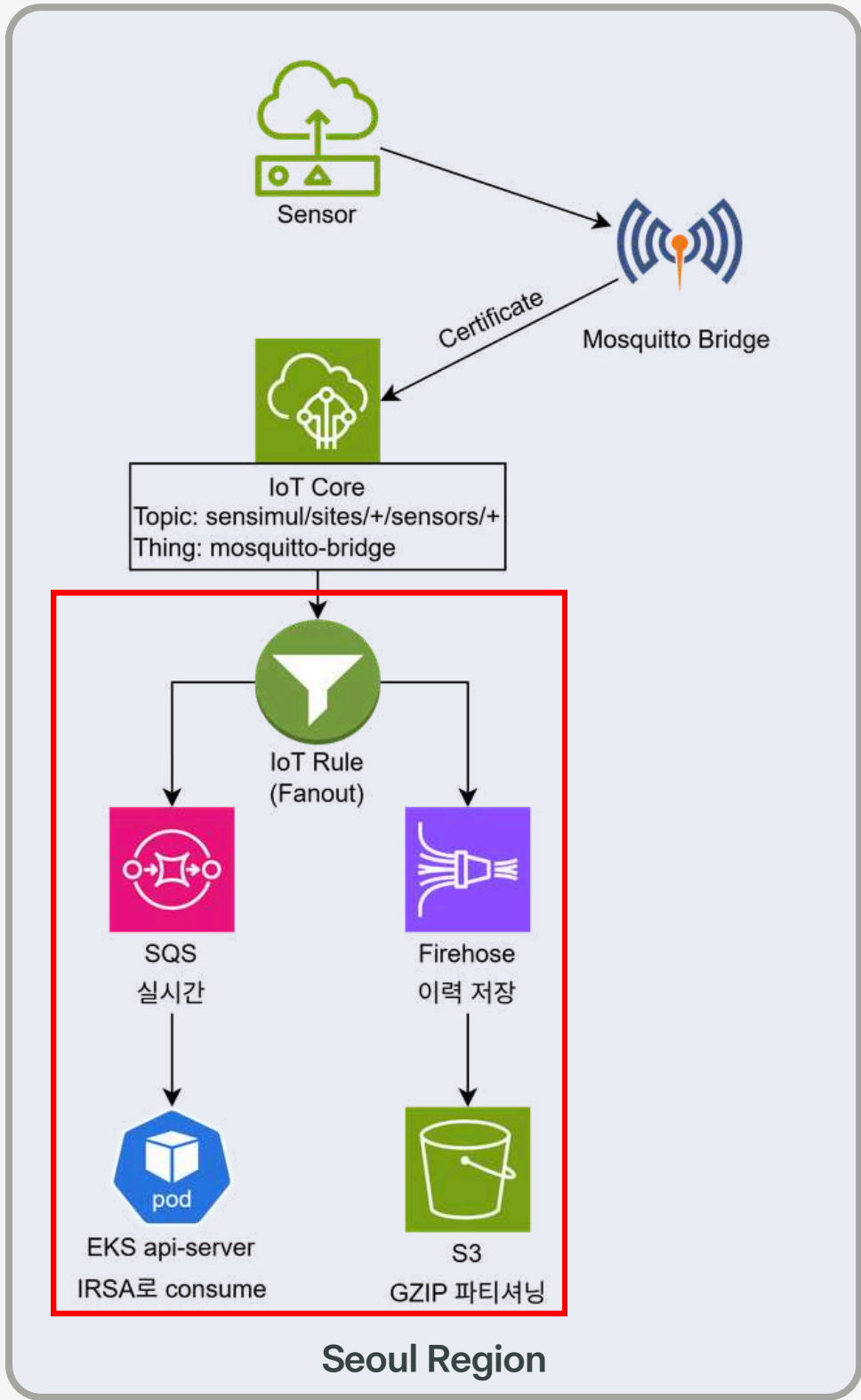
2 Backend

빌드와 배포를 분리한 경로

- 서비스별 1회 빌드 후 동일 SHA 태그로 두 리전 ECR에 push해 양 리전이 같은 이미지를 사용함
- GitHub Actions는 클러스터에 직접 kubectl을 실행하지 않고, GitOps 레포의 이미지 태그를 커밋 SHA로 갱신하는 것까지만 맡음
- 각 클러스터의 ArgoCD가 레포 변경을 감지해 Auto Sync함
- 매니페스트와 어긋나면 selfHeal이 되돌리고, 지운 리소스는 prune이 정리함

IoT 파이프라인 구성

온프레미스 센서 → IoT Core → 실시간(SQS)·이력(Firehose) 이중 경로



IoT Thing

사물 (1) 정보

IoT는 클라우드에 있는 물리적 장치를 디지털로 표현한 것입니다. 기본 검색 또는 플릿 인덱싱을 집계를 실행하고, 사물 연결 상태를 쿼리할 수도 있습니다.

☐ 이름

☐ [mosquitto-bridge](#)

IoT Policy

```
"Statement": [
  {
    "Action": "iot:Connect",
    "Effect": "Allow",
    "Resource": "arn:aws:iot:ap-northeast-2:448768137813:client/mosquitto-bridge-seoul"
  },
  {
    "Action": "iot:Publish",
    "Effect": "Allow",
    "Resource": "arn:aws:iot:ap-northeast-2:448768137813:topic/sensimul/sites/*"
  }
],
"Version": "2012-10-17"
```

IoT Rules

작업 (2)

이벤트가 트리거될 때 작업이 발생합니다. 작업은 모든 작업이 완료되거나 오류가 발생할 때까지 실행됩니다. 작업을 추가하거나 제거하려면 규칙을 편집해야 합니다.

서비스	작업
☐ Simple Queue Service (SQS)	메시지를 SQS 대기열로 전송
☐ Data Firehose 스트림	Data Firehose 스트림으로 메시지 전송

Active-Standby

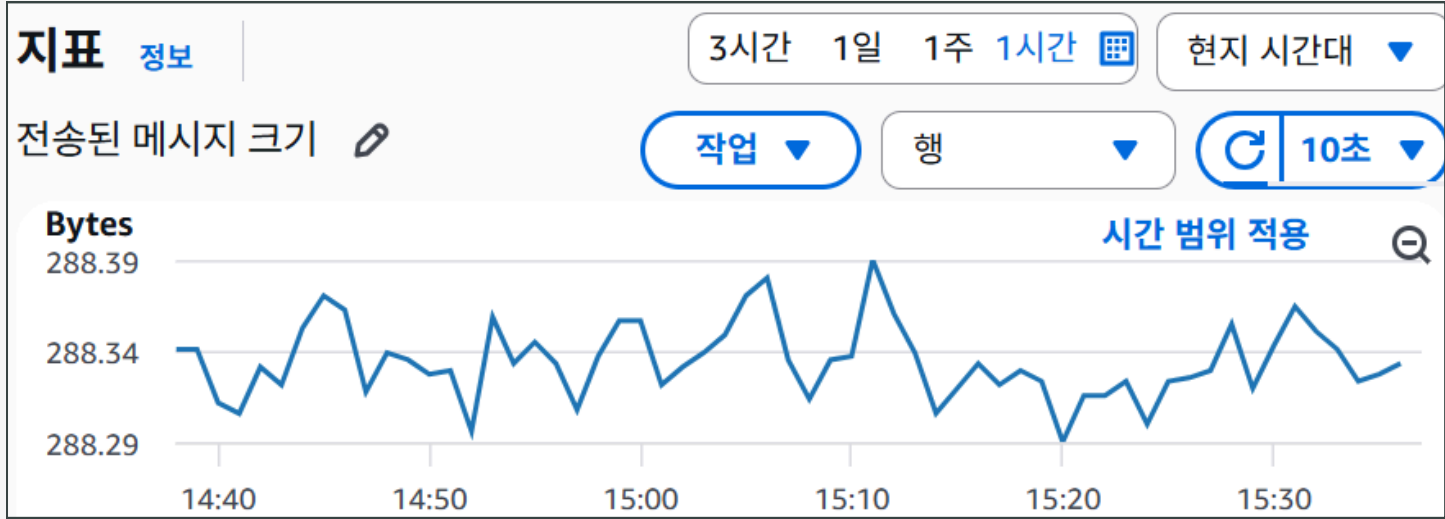
	Seoul	Ohio
IoT Rule	SQS + Firehose	SQS
목적	실시간 + 이력	Failover 대기

02 구현

센서 데이터 실시간 처리·이력 저장

실시간 처리와 이력 분석의 요구가 달라 경로 분리

SQS → Backend (실시간 처리)



- 전달 흐름
- 1. SQS 메시지 수신
 - 2. Backend 처리
 - 3. 대시보드 실시간 반영

환경 모니터링

실시간 센서, 알림, 히스토리, 제어기 명령 상태를 확인하세요.

등록 센서
19
활성 19대

정상 알림
18
최근 30일 기준

주의 알림
0
경고 상태 확인 필요

위험 알림
1
즉시 조치 필요

최근 최고 우선순위 알림
서울 강서 냉동실 온도 센서: 30.874004465785255celsius (normal)
서울 강서 냉동실 온도 센서 • 2026. 06. 15. 오후 05:09

Data Firehose → S3 (이력 저장)

Amazon S3 > 버킷 > stockops-sensor-data > sensors/ > year=2026/ > month=06/ > day=18/

day=18/

객체 속성

객체 (13)

객체는 Amazon S3에 저장되어 있는 기본 엔터티입니다. [Amazon S3 인벤토리](#)를 사용하여 버킷에 있는 모든 객체의 목록을 얻을 수 있습니다. 다른 사용자가 객체에 액세스할 수 있게 하려면 명시적으로 권한을 부여해야 합니다. [자세히](#)

접두사로 객체 찾기

버전 표시

☐	이름	▲	유형	▼	마지막 수정	▼	크기	▼	스토리지 클래스
☐	stockops-sensor-history-1-2026-06-18-00-37-36-ad431cf9-7d7e-4834-b2b9-7bb180eea2be.gz		gz		2026. 6. 18. am 9:52:37 AM KST		91.1KB		Standard
☐	stockops-sensor-history-1-2026-06-18-00-52-37-d7db8097-5254-49a7-a6d3-e44284283111.gz		gz		2026. 6. 18. am 10:07:38 AM KST		90.5KB		Standard
☐	stockops-sensor-history-1-2026-06-18-01-07-32-22a74871-ddb2-47dd-ac69-007764e2dc86.gz		gz		2026. 6. 18. am 10:22:38 AM KST		90.8KB		Standard

buffering_interval = 900

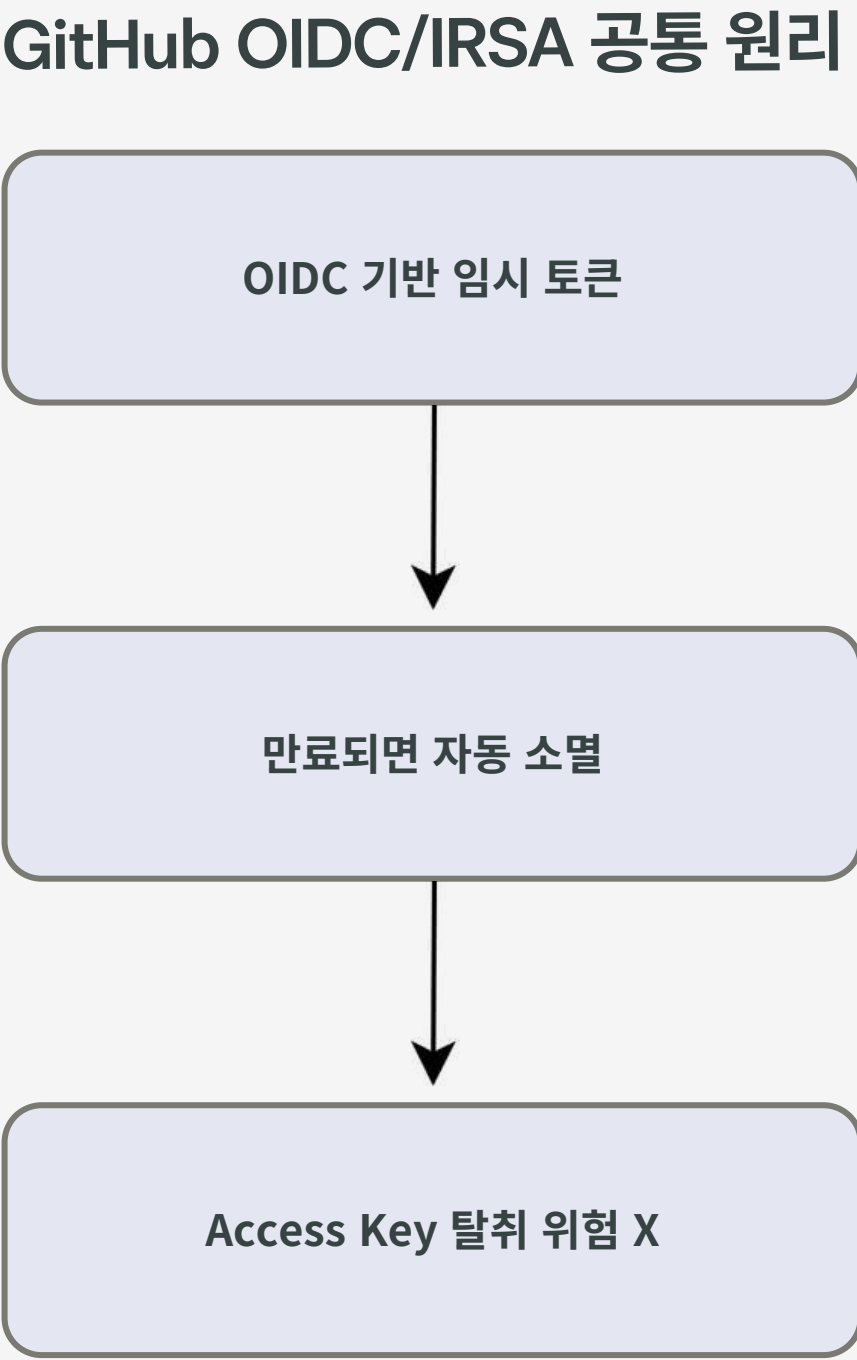
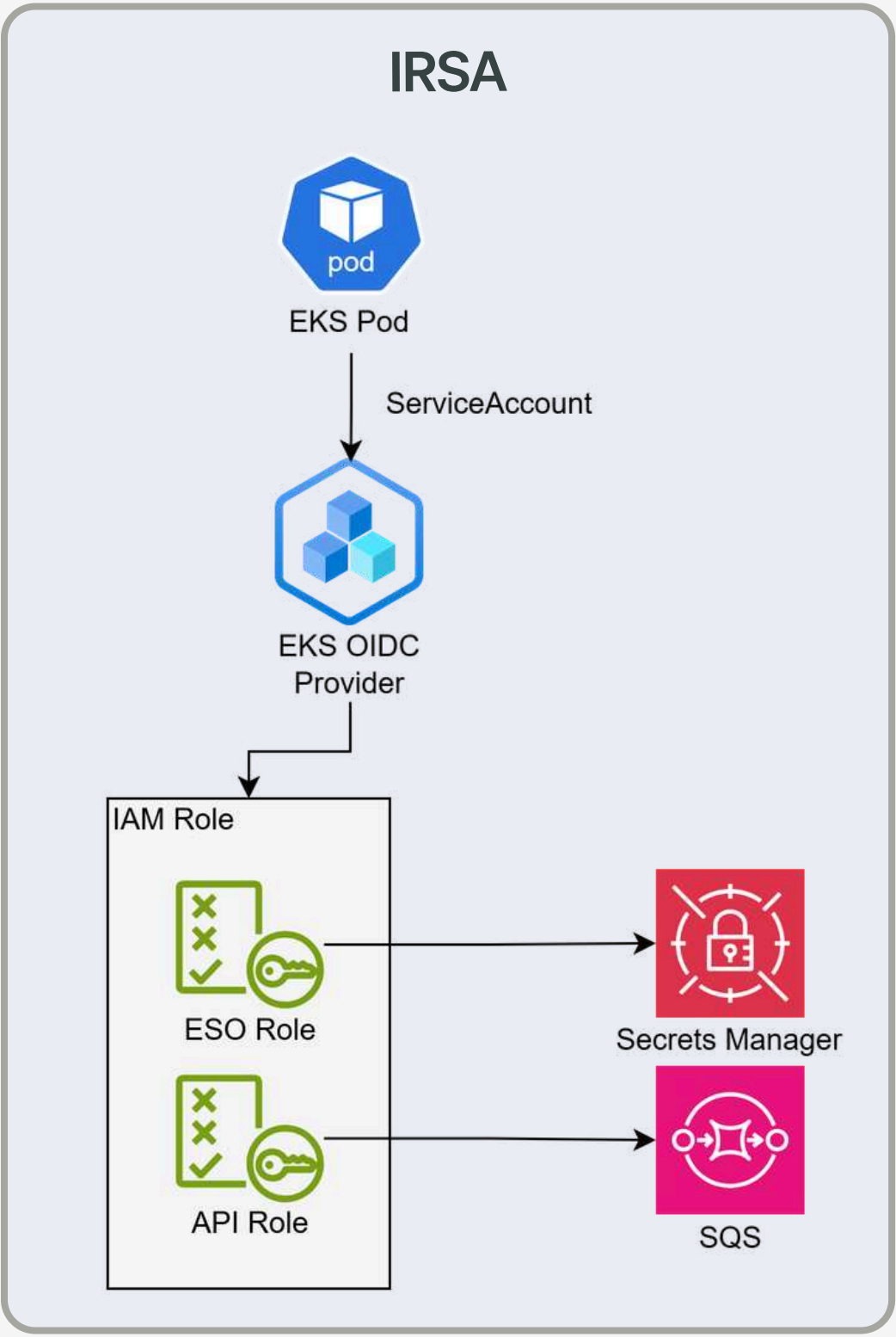
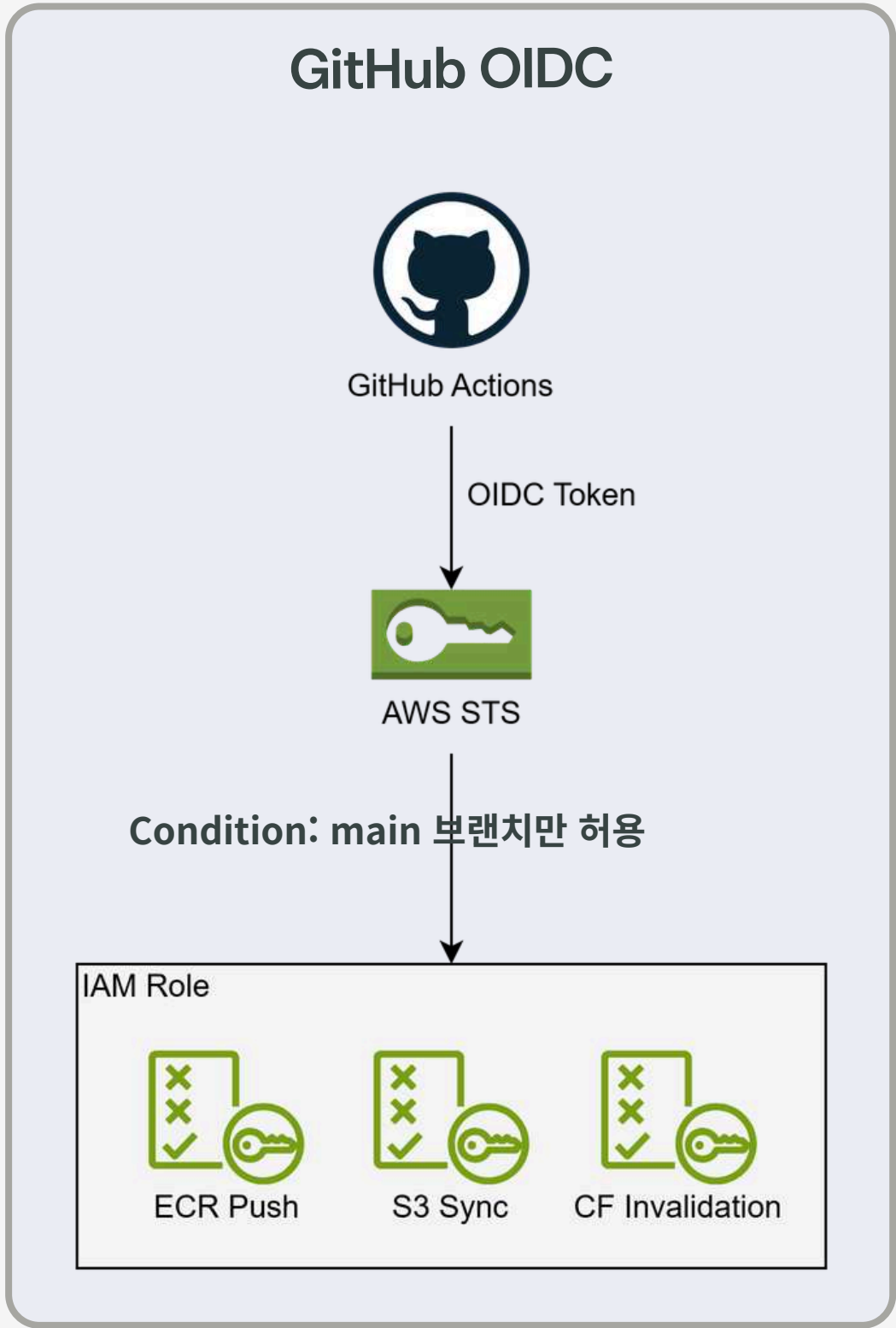
DataFreshness = 15분

센서 데이터가 지속 유입되며 buffering_interval 900초 설정 대로 15분마다 S3에 배치 저장됨



키리스 인증

GitHub OIDC·IRSA 적용 — 장기 액세스 키 0개



시크릿 관리

Secrets Manager + ESO 동기화 — 매니페스트에 시크릿 미포함

Secrets

보안 암호 이름

▼

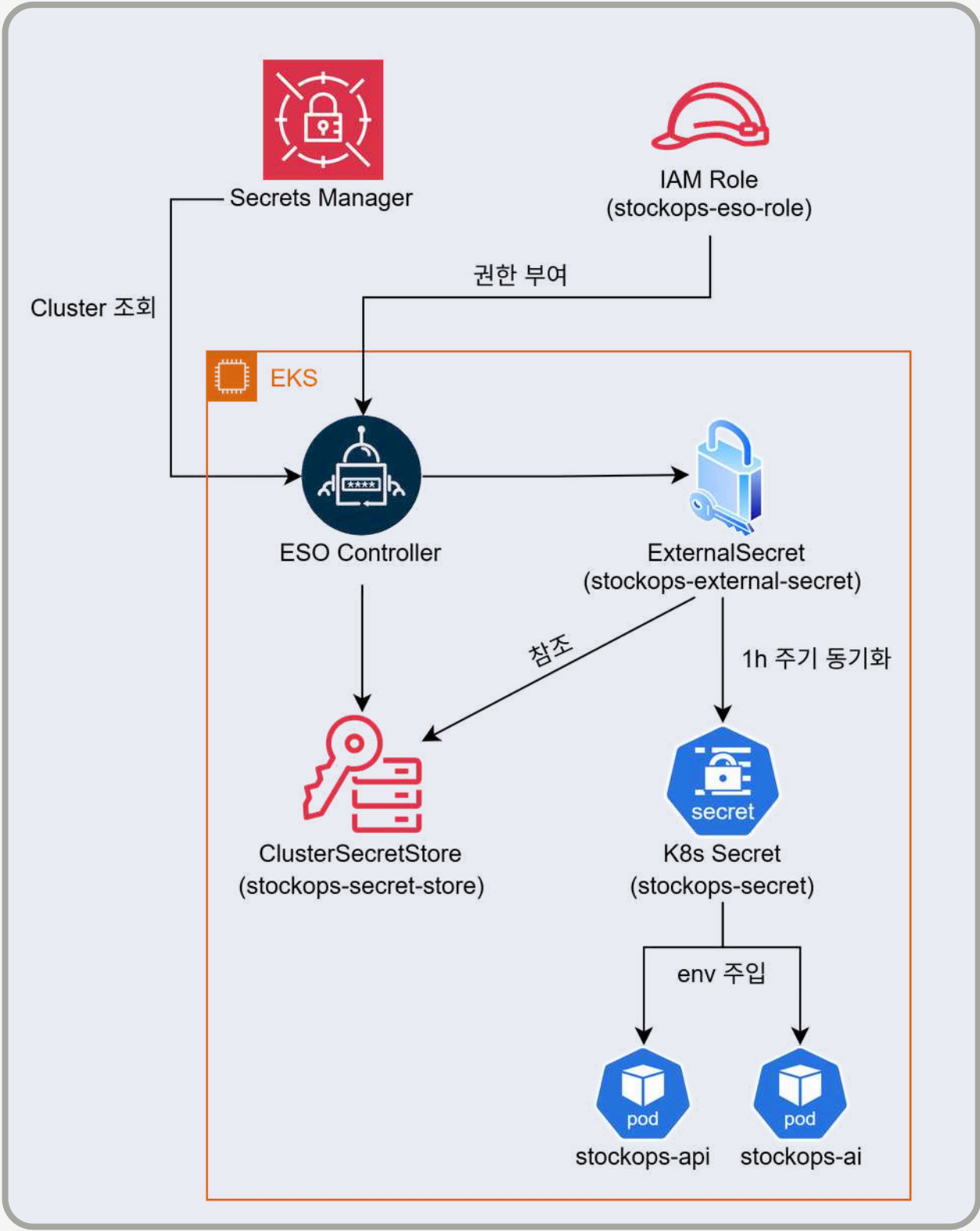
stockops/app

Secrets Key & Value

키/값		일반 텍스트
보안 암호 키	보안 암호 값	
AI_MODULE_API_KEY		
DB_PASSWORD		
DB_USERNAME		
GEMINI_API_KEY		
JWT_SECRET		
SPRING_MAIL_PASSWORD		
STOCKOPS_AI_SERVICE_API_KEY		
STOCKOPS_BEDROCK_AGENT_ALIAS_ID		
STOCKOPS_BEDROCK_AGENT_ID		
STOCKOPS_BEDROCK_AWS_ACCESS_KEY_ID		
STOCKOPS_BEDROCK_AWS_SECRET_ACCESS_KEY		
STOCKOPS_BEDROCK_GUARDRAIL_ID		
STOCKOPS_BEDROCK_KNOWLEDGE_BASE_ID		
STOCKOPS_VERTEX_AI_CREDENTIALS_JSON		
STOCKOPS_VERTEX_AI_PROJECT_ID		

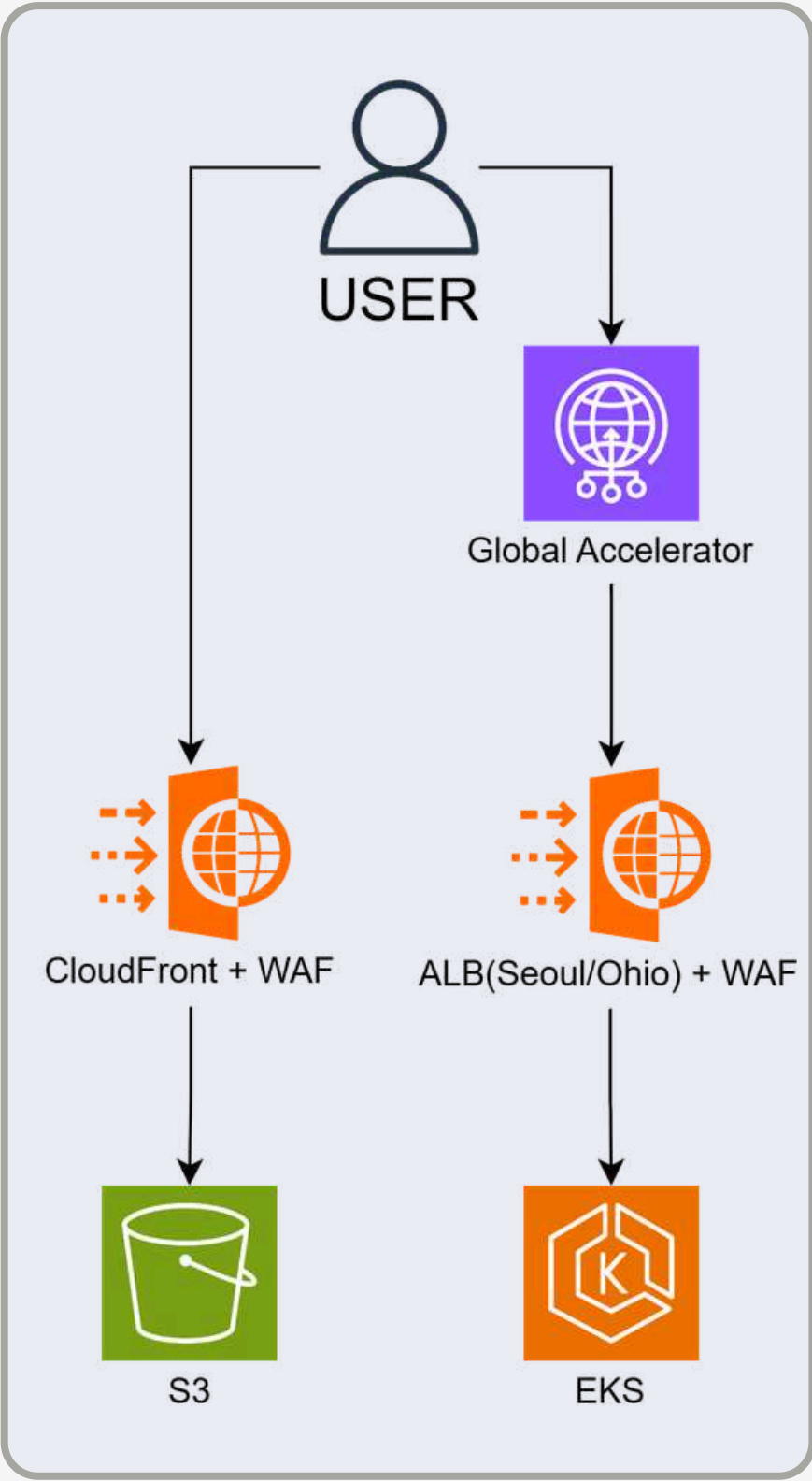
```
PS C:\KJW\Team_Project\Stockops-Infra\regions\seoul> kubectl get externalsecret -n stockops
```

NAME	STORETYPE	STORE	REFRESH INTERVAL	STATUS	READY	LAST SYNC
stockops-external-secret	ClusterSecretStore	stockops-secret-store	1h	SecretSynced	True	34m



경계 방어 – 애플리케이션 계층 (WAF)

엣지·리전 2계층 WAF — 광범위 룰셋은 Count로 오탐 관찰, 커스텀 RateLimit은 Block



CloudFront WAF Rules

stockops-cloudfront-waf에 대한 규칙 관리		×
< stockops-cloudfront-waf		
규칙은 규칙 우선 순위 오름차순으로 표시됩니다.		
AWSManagedRulesCommonRuleSet	Count 모드에서 실행 중	700 WCU >
AWSManagedRulesAmazonIpReputationList		25 WCU >

ALB WAF Rules

seoul-alb-waf에 대한 규칙 관리		×
< seoul-alb-waf		
규칙은 규칙 우선 순위 오름차순으로 표시됩니다.		
AWSManagedRulesCommonRuleSet	Count 모드에서 실행 중	700 WCU >
AWSManagedRulesKnownBadInputsRuleSet		200 WCU >
AWSManagedRulesSQLiRuleSet	Count 모드에서 실행 중	200 WCU >
LoginRateLimit	Custom Rule	14 WCU >
RateLimit		2 WCU >

1 CloudFront

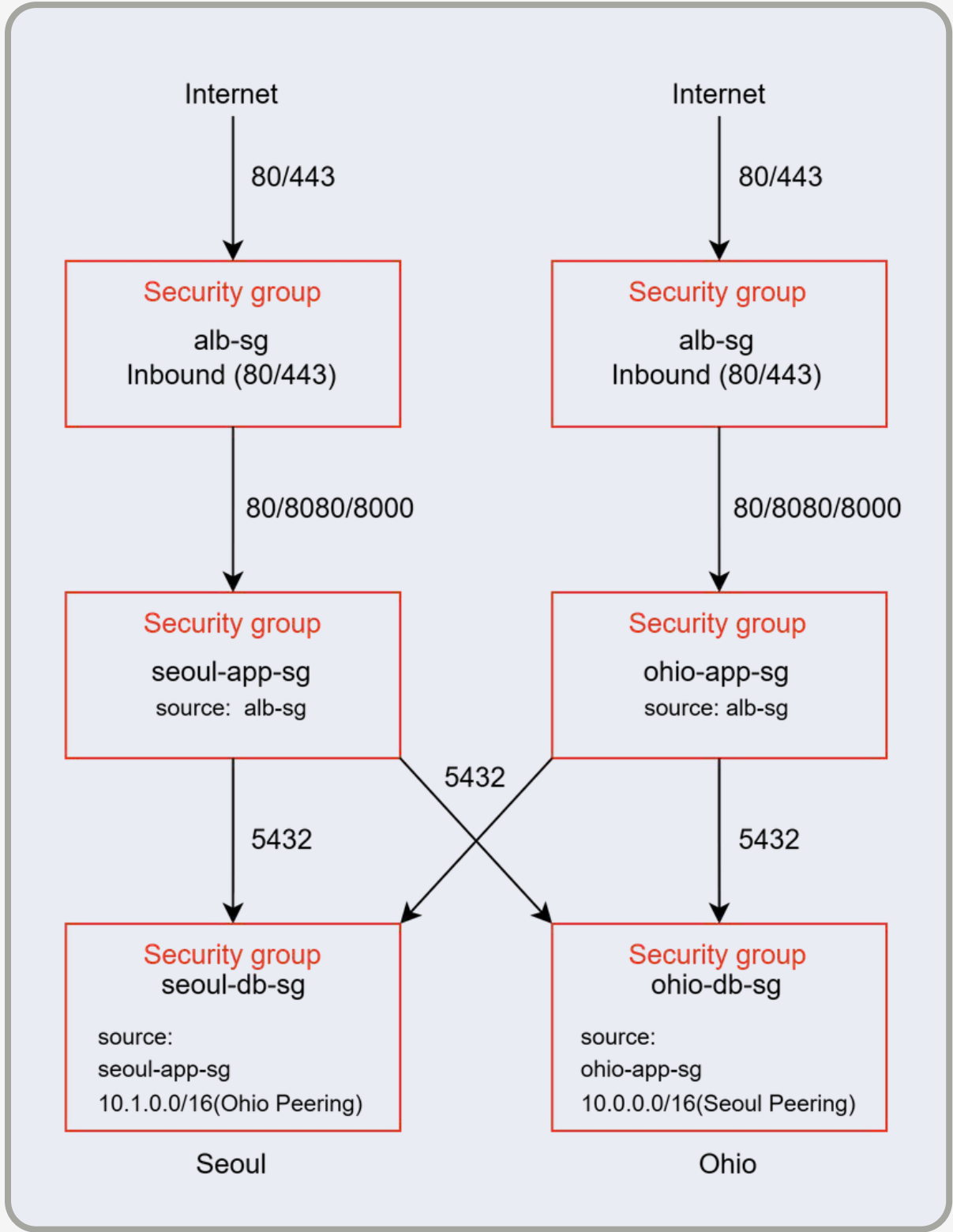
- AWS ManagedRulesCommonRuleSet**
- XSS, 악성 파일 업로드, HTTP 프로토콜 위반 등을 탐지
- AWSManagedRulesAmazonIpReputationList**
- AWS가 수집한 악성 IP 데이터베이스(봇, 스캐너, 해킹 시도 이력 IP)와 대조해 매칭되면 즉시 차단

2 ALB

- AWSManagedRulesCommonRuleSet**
- CloudFront와 동일
- AWSManagedRulesKnownBadInputsRuleSet**
- Log4Shell, Spring4Shell 등 알려진 취약점 익스플로잇 패턴을 즉시 차단
- AWSManagedRulesSQLiRuleSet**
- SQL Injection 패턴 전용 룰셋
- LoginRateLimit**
- /api/v1/auth/login 경로로 IP당 5분에 100회 초과 시 즉시 차단
- RateLimit**
- 전체 경로 대상으로 IP당 5분에 2000회 초과 시 차단

경계 방어 – 네트워크 계층 (SG)

IP 대역이 아닌 SG ID를 source로 지정 — alb-sg → app-sg → db-sg 단방향 체인



설계 원칙

최소 권한	포트 최소화
- ALB SG ID를 source로 직접 지정 - IP 대역이 아닌 SG 참조 - 동일 VPC 내 불필요한 접근 차단	- ALB → EKS 구간 서비스별로 분리 (80/8080/8000) - 포트 범위 대신 개별 규칙으로 명시
RDS 격리	크로스 리전 접근 Peering
- 인터넷 → RDS 경로 제거 - app-sg + Peering CIDR만 인바운드 트래픽 허용	- Seoul → Ohio Replica - 읽기 부하 분산 - Ohio → Seoul Master - RDS 복제 동기화

트러블슈팅

증상이 드러난 곳과 원인이 있던 곳이 달랐던 3건

증상	원인	해결	결과
한쪽 리전을 다시 적용하면 다른 쪽 피어링 경로가 삭제됨	리전 VPC 모듈이 라우팅 테이블을 통째로 덮어씀	적용 순서를 규칙으로 문서화 (Seoul → Ohio → Peering → Global) + 하드 코딩 참조를 remote_state 출력값으로 교체	전체 삭제 후 재적용해도 동일 구성 재현 확인
부하가 몰리자 파드가 Pending 상태로 멈춤	Karpenter의 SecurityGroupSelectorTerms가 참조하는 태그가 클러스터 보안 그룹에 누락	aws_ec2_tag 리소스로 태그 보강	30초 내 스팟 노드 자동 프로비저닝 (t3.large ×2)
IoT 기기는 발행하는데 DB 테이블이 비어 있음	IRSA 권한 · 환경변수 · JSON 매핑 (@JsonProperty) 누락이 겹침	권한 → 설정 → 코드 → 데이터 순으로 계층별 순차 교정	IoT Core → SQS → RDS 실시간 경로 복원

성과

전체 인프라를 코드로 정의하고 삭제 후 재생성까지 검증 완료

1 노드 자원 부족 해소

운영 파드 5개 → 3개 (40% 감축)
트래픽과 무관하게 상시 점유하던 자원 제거
프론트 수정 시 서버 재배포 불필요

2 삭제 후 재생성 검증

4개 state 전부 동일 구성으로 복원
apply·destroy 전 주기를 규칙으로 고정
동일 모듈로 서울·오하이오 두 리전 구성

3 장기 자격증명 미사용

파이프라인·파드 인증에 장기 키 0개
GitHub OIDC·IRSA로 매번 임시 자격증명 발급
시크릿이 코드·매니페스트에 남지 않음

4 배포 무인화

Merge 이후 배포까지 수동 단계 없음
동일 커밋 SHA로 서울·오하이오 동시 갱신
필요 시 리전별 독립 롤백 가능

감사합니다.

Name	김진우
Email	kkjjww1503@naver.com
Phone	010-3470-1503
GitHub	Infra → https://github.com/jinuuuKim/Stockops-Infra GitOps → https://github.com/jinuuuKim/Stockops-GitOps